DevOps and Cloud Computing Fundamentals

COMPREHENSIVE PRACTICAL ASSIGNMENT

Sakila Flask Application — End-to-End DevOps Pipeline

Repository: https://github.com/20030054/spring26_sakila_app

Spring 2026 — Assignment Handout

Total Marks: 100 | Due Date: 25-04-2026

Student Name: _____ Roll Number: _____

Section: _____

1. Assignment Overview

This assignment is designed as a capstone practical exercise that integrates every major topic covered in the DevOps and Cloud Computing Fundamentals course. You will work with a real-world Flask web application backed by a MySQL database (the Sakila DVD Rental database) and will be required to demonstrate mastery across three pillars: Git Version Control, Docker Containerization, and CI/CD with GitHub Actions.

Unlike a step-by-step tutorial, this assignment presents you with scenarios, requirements, and deliberate challenges that require you to apply your knowledge, troubleshoot problems, read documentation, and make design decisions independently. Simply copying commands from the lecture notes will not be sufficient.

ACADEMIC INTEGRITY WARNING

All work must be your own. Submissions will be checked for plagiarism against each other and online sources.

You must include screenshots with your terminal showing your unique username/hostname as proof of original work.

Any two submissions with identical screenshots, commit hashes, or container IDs will receive zero marks.

1.1 Learning Objectives

- Demonstrate proficiency in Git workflows including branching, merging, conflict resolution, and pull requests
- Build, optimize, and troubleshoot Docker images using Dockerfiles with multi-stage builds
- Architect multi-container applications using Docker networks and volumes for persistent data
- Author Docker Compose files to orchestrate complex application stacks
- Design and implement CI/CD pipelines using GitHub Actions with multiple stages and conditional logic
- Debug and resolve intentionally planted errors in configuration files and code
- Document technical decisions and justify architectural choices in writing

1.2 Marking Rubric Summary

Section	Task Area	Marks	Weight
Part A	Git Mastery & Collaboration Workflow	25	25%

Part B	Docker Containerization & Architecture	35	35%
Part C	CI/CD Pipeline with GitHub Actions	25	25%
Part D	Reflection & Architecture Documentation	15	15%
	TOTAL	100	100%

1.3 Prerequisites & Tools Required

- A GitHub account (free tier is sufficient)
- Docker Desktop installed and running (v20.10 or later)
- Git CLI installed (v2.30 or later)
- A code editor (VS Code recommended)
- Terminal/command-line access (Bash, PowerShell, or Zsh)
- Stable internet connection for pulling Docker images

1.4 Submission Requirements

- **GitHub Repository Link:** Your forked repository URL with all branches, commits, and pull requests visible.
- **Screenshots Document:** A PDF containing labeled screenshots proving each task was completed. Each screenshot MUST show your terminal prompt with your unique username.
- **Written Report:** A 2–3 page document for Part D (Reflection & Architecture) in PDF format.
- **All files:** Dockerfile, docker-compose.yml, and .github/workflows/ YAML files must be committed and pushed to your repository.

2. PART A — Git Mastery & Collaboration Workflow [25 Marks]

This section tests your ability to use Git beyond the basics. You will fork the repository, create a structured branching model, intentionally create and resolve merge conflicts, and simulate a collaborative workflow using pull requests.

Task A1: Repository Setup & Branching Strategy [5 Marks]

Scenario: Your team lead has decided that the Sakila project must follow a structured Git branching model. You are responsible for setting up the repository with a proper branching strategy before any development begins.

Requirements:

1. Fork the repository https://github.com/20030054/spring26_sakila_app to your own GitHub account.
2. Clone your fork locally and verify the remote configuration.
3. Create the following branch structure from the main branch: develop (integration branch), feature/docker-optimization (for Part B Docker work), feature/ci-pipeline (for Part C CI/CD work), and hotfix/bugfix (for fixing an intentional bug described below).
4. Set up branch protection rules on your main branch: require at least 1 pull request review before merging (you can use a second GitHub account or configure appropriately), and require status checks to pass (this will be configured in Part C).

DELIVERABLES FOR A1

Screenshot of your fork showing your GitHub username in the URL.
Output of: git remote -v
Output of: git branch -a (showing all branches)
Screenshot of branch protection rules configuration on GitHub.

Task A2: Intentional Merge Conflict Creation & Resolution [8 Marks]

Scenario: Two developers are working simultaneously on different features but have modified the same files. You must simulate this conflict and demonstrate a clean resolution strategy.

Requirements:

5. Switch to the develop branch. Create a new branch called feature/update-config from develop.
6. On the feature/update-config branch, modify the config.py file as follows: change the default MYSQL_HOST from 'mysql-container' to 'sakila-db-server', add a new configuration variable CONNECTION_TIMEOUT = int(os.environ.get('CONNECTION_TIMEOUT', '30')), and add a comment block at the top of the file with your name and date.
7. Commit your changes with a descriptive, conventional commit message (e.g., feat(config): update database host and add timeout setting).
8. Switch back to develop. Create ANOTHER branch called feature/add-healthcheck from develop.
9. On feature/add-healthcheck, modify the SAME config.py file: change the default MYSQL_HOST to 'db-primary', add a variable HEALTH_CHECK_INTERVAL = int(os.environ.get('HEALTH_CHECK_INTERVAL', '10')), and add a DIFFERENT comment block at the top with a team member name and date.
10. Commit these changes.
11. Now merge feature/update-config into develop first (this should succeed cleanly).
12. Then attempt to merge feature/add-healthcheck into develop. This WILL produce a merge conflict.
13. Resolve the merge conflict manually using your editor (NOT a merge tool). Your final resolution must: keep MYSQL_HOST as 'sakila-db-server' (from the first branch), include BOTH new configuration variables (CONNECTION_TIMEOUT and HEALTH_CHECK_INTERVAL), and combine both comment blocks into a single, clean header comment.
14. Complete the merge commit with a message that explains what was conflicting and how you resolved it.

CRITICAL REQUIREMENT

You MUST show the conflict markers (<<<<<<, ======, >>>>>>) in a screenshot BEFORE resolving them.
You MUST show the resolved file AFTER conflict resolution.
You MUST show the git log --graph --oneline output after the merge is complete.

DELIVERABLES FOR A2

Screenshot showing the conflict markers in config.py
Screenshot showing the resolved config.py
Output of: git log --graph --oneline --all (at least 10 commits visible)
The conflict resolution commit message

Task A3: Pull Request Workflow with Code Review Simulation [7 Marks]

Scenario: Before your Docker changes can go into production, they must go through a proper code review process via pull requests. You will create a PR, add reviewers, respond to simulated review feedback, and merge using a squash merge strategy.

Requirements:

15. Push your develop branch (with the resolved conflicts) to your remote fork.
16. Create a Pull Request from develop to main on GitHub.
17. In the PR description, write a comprehensive summary that includes: a bulleted list of all changes made, a section explaining the conflict that was resolved and the rationale for your resolution choices, and a testing checklist (checkboxes) listing what you verified.
18. Add at least 3 meaningful inline comments on your own PR reviewing your code (simulate a self-review — point out potential issues, suggest improvements, or explain complex logic).
19. After adding the review comments, push at least one additional commit to develop that addresses one of your review comments (e.g., adding a docstring, fixing a typo, improving a variable name).
20. Merge the PR using the Squash and Merge strategy. Write a clean squash commit message summarizing all changes.

DELIVERABLES FOR A3

Screenshot of the PR description (full content visible).
Screenshot of at least 3 inline review comments.
Screenshot showing the additional commit addressing a review comment.
Screenshot of the successful squash merge with the commit message.

Task A4: Upstream Sync & Tagging [5 Marks]

Scenario: The original repository has received updates. You must configure your fork to track the upstream repository, fetch upstream changes, rebase your work on top, and create a semantic version tag.

Requirements:

21. Add the original repository as an upstream remote.

```
git remote add upstream
https://github.com/20030054/spring26_sakila_app.git
```

22. Fetch upstream changes and show the difference between your main and upstream/main.
23. If there are upstream changes, rebase your main on top of upstream/main (if no upstream changes exist, document the commands you would use and explain the difference between git merge and git rebase in your report).
24. Create an annotated tag v1.0.0 on your main branch with the message "Assignment Part A Complete — Git Workflow Established".
25. Push the tag to your remote fork.

DELIVERABLES FOR A4

Output of: git remote -v (showing both origin and upstream)
Output of: git log --oneline upstream/main..main (or explanation if no diff)
Output of: git tag -l -n1 (showing the annotated tag)
Output of: git log --oneline --decorate (showing the tag on the commit)

3. PART B — Docker Containerization & Architecture [35 Marks]

This section challenges you to go beyond simply running Docker commands. You will analyze, debug, optimize, and architect a multi-container application using Docker best practices.

Task B1: Dockerfile Analysis & Debugging [7 Marks]

Scenario: A junior developer wrote a Dockerfile for the Sakila app, but it contains several issues and inefficiencies. You must identify the problems, fix them, and optimize the image.

The Problematic Dockerfile (create this as Dockerfile.broken in your repo):

```
FROM python:3.9

WORKDIR /app

COPY . /app

RUN pip install flask
RUN pip install pymysql
RUN pip install cryptography

ENV MYSQL_HOST=localhost
ENV MYSQL_USER=root
ENV MYSQL_PASSWORD=supersecretpassword
ENV MYSQL_DB=sakila

EXPOSE 5000
EXPOSE 3306
EXPOSE 22

CMD ["python", "app.py"]
```

Requirements:

26. Identify and document AT LEAST 6 distinct issues with the Dockerfile above. For each issue, explain: what the problem is, why it matters (security, performance, caching, best practice), and how to fix it.

27. Create a corrected and optimized Dockerfile that addresses all identified issues. Your optimized Dockerfile must: use an appropriate slim or alpine base image, leverage Docker layer caching correctly, use a requirements.txt file for dependency installation, NOT hardcode sensitive information, expose only the necessary port, include a HEALTHCHECK instruction, use a non-root user for running the application, and include appropriate labels (maintainer, version, description).
28. Build both images (broken and fixed) and compare their sizes using docker images. Document the size difference.

CHALLENGE ELEMENT

Your optimized Dockerfile must produce an image that is at least 40% smaller than the broken version.

If your image is not at least 40% smaller, explain what further optimizations could be applied.

DELIVERABLES FOR B1

Dockerfile.broken committed to your repo

The corrected Dockerfile committed to your repo

A written analysis document (in your report) listing all 6+ issues found

Screenshot of: docker images (showing both image sizes side by side)

Screenshot of: docker inspect <your-image> (showing labels and healthcheck)

Task B2: Container Networking Deep Dive [8 Marks]

Scenario: You must set up the complete application stack manually using Docker CLI commands (no Docker Compose yet). You must demonstrate understanding of how container networking works by completing a series of networking challenges.

Requirements:

29. Create a custom bridge network called sakila-net with the subnet 172.20.0.0/16 and gateway 172.20.0.1.

```
# Hint: Use docker network create with --subnet and --gateway flags
```

30. Run the MySQL container on this network with a STATIC IP address of 172.20.0.10. Name it sakila-db.
31. Load the Sakila database into the MySQL container (download and import the schema and data SQL files).
32. Run the Flask application container on the same network. Do NOT assign a static IP — let Docker assign it dynamically. Name it sakila-app.
33. Verify connectivity by executing the following from INSIDE the Flask container:

```
# Install ping inside the container first
docker exec sakila-app apt-get update && docker exec sakila-app apt-get
install -y iputils-ping


# Ping MySQL by container name
docker exec sakila-app ping -c 3 sakila-db


# Ping MySQL by static IP
docker exec sakila-app ping -c 3 172.20.0.10


# Test MySQL connectivity from Flask container
docker exec sakila-app python -c "import pymysql;
c=pymysql.connect(host='sakila-
db',user='root',password='admin',db='sakila'); print('SUCCESS');
c.close()"
```

34. Inspect the network and document which containers are attached, their IP addresses, and the network driver.
35. CHALLENGE: Demonstrate what happens when you try to connect two containers on DIFFERENT networks. Create a second network called sakila-isolated, run a test container on it, and show that it CANNOT reach sakila-db.

DELIVERABLES FOR B2

All Docker commands used (in order)
Screenshot of: docker network inspect sakila-net (showing both containers)
Screenshot of successful ping and Python connectivity test
Screenshot proving cross-network isolation failure
Output of: docker exec sakila-app cat /etc/hosts

Task B3: Docker Volumes & Data Persistence [7 Marks]

Scenario: Your manager wants proof that the database survives container destruction. You must demonstrate persistent storage and explain the differences between volume types.

Requirements:

36. Create a named volume called sakila-mysql-data.
37. Run the MySQL container using this named volume mounted to /var/lib/mysql.
38. After the Sakila database is loaded, execute a custom INSERT to create a "fingerprint" record that proves your data is unique:

```
docker exec -it sakila-db mysql -uroot -padmin -e "USE sakila;
INSERT INTO actor (first_name, last_name) VALUES ('YOUR_FIRST_NAME',
'YOUR_LAST_NAME');"
```

39. Verify the record exists:

```
docker exec -it sakila-db mysql -uroot -padmin -e "SELECT * FROM
sakila.actor ORDER BY actor_id DESC LIMIT 5;"
```

40. DESTROY the MySQL container completely (docker rm -f sakila-db).
41. Create a BRAND NEW MySQL container using the same named volume.
42. Verify that your fingerprint record STILL EXISTS after container recreation.
43. CHALLENGE: Now demonstrate the OPPOSITE — create a MySQL container WITHOUT a named volume (using the container's default filesystem), insert a different fingerprint record, destroy the container, recreate it, and show that the data is LOST.
44. Inspect the volume to find its physical location on the host machine:

```
docker volume inspect sakila-mysql-data
```

CRITICAL GRADING NOTE

Your fingerprint record MUST contain your actual name (matching your submission identity). Screenshots must show the same unique actor_id before destruction and after recreation.

DELIVERABLES FOR B3

Screenshot showing the INSERT of your fingerprint record
Screenshot showing the record EXISTS after container destruction and recreation
Screenshot showing data LOSS when no volume is used
Output of: docker volume inspect sakila-mysql-data
Output of: docker volume ls

Task B4: Docker Compose Orchestration [13 Marks]

Scenario: Manual container management is tedious and error-prone. You must create a comprehensive Docker Compose file that automates the entire stack setup with a single command. But the requirements are more complex than a simple two-container setup.

Requirements — Create a docker-compose.yml file with the following services:

Service 1: db (MySQL 8.0)

- Named volume for data persistence

- Custom network (sakila-network)
- Health check that verifies MySQL is ready to accept connections using mysqladmin ping
- Environment variables for root password, database name
- Restart policy: unless-stopped
- Resource limits: memory limit of 512MB

Service 2: app (Flask Application)

- Build from the local Dockerfile (your optimized version)
- Depends on db service with a condition of service_healthy
- Port mapping 5000:5000
- Environment variables referencing the db service name as MYSQL_HOST
- Same custom network as db
- Restart policy: on-failure with max retry of 5

Service 3: db-admin (phpMyAdmin or Adminer)

- Use the adminer:latest image
- Port mapping 8080:8080
- Depends on db service
- Same custom network
- This gives you a web-based database management UI at http://localhost:8080

Service 4: test-runner (a one-time container)

- Build from the same Dockerfile as the app
- Override the command to run: python -m pytest tests/ -v
- Depends on db being healthy
- Same network
- Profile: test (so it only runs when explicitly invoked with --profile test)

Additional Docker Compose Requirements:

45. Define a custom bridge network named sakila-network in the networks section.
46. Define the named volume sakila-db-data in the volumes section.
47. Use an .env file to store all sensitive values (passwords, database names). Your docker-compose.yml should reference these using ${VARIABLE_NAME} syntax. Include a .env.example file with placeholder values.

48. Demonstrate the full lifecycle:

```
# Start the full stack
docker compose up -d

# Verify all services are running
docker compose ps

# View aggregated logs
docker compose logs

# Run the test suite
docker compose --profile test run test-runner

# Scale test (optional, for bonus understanding):
# Explain in your report WHY you cannot scale the db service

# Tear down everything
docker compose down

# Tear down INCLUDING volumes (destructive)
docker compose down -v
```

CHALLENGE ELEMENTS

The depends_on with condition: service_healthy requires a properly configured healthcheck on the db service. If your healthcheck is wrong, the app will never start. Debug this carefully.

The test-runner service uses Docker profiles. If you don't configure profiles correctly, the test container will start with every 'docker compose up' and immediately crash (because the tests directory might not have all tests passing).

You must explain in your report what happens differently between 'docker compose down' and 'docker compose down -v'.

DELIVERABLES FOR B4

docker-compose.yml file committed to your repository

.env.example file committed (NOT the actual .env file — add .env to .gitignore!)

Screenshot of: docker compose up -d output

Screenshot of: docker compose ps (showing all services healthy/running)

Screenshot of the Flask app running at http://localhost:5000

Screenshot of Adminer running at http://localhost:8080

Screenshot of: docker compose --profile test run test-runner output
Screenshot of: docker compose down -v output

4. PART C — CI/CD Pipeline with GitHub Actions [25 Marks]

This section requires you to design and implement a comprehensive CI/CD pipeline that automates testing, building, security scanning, and deployment preparation for the Sakila application.

Task C1: Multi-Stage CI Pipeline [12 Marks]

Scenario: Your organization requires an automated pipeline that runs on every push and pull request. The pipeline must include multiple stages that run in a defined order with proper dependency management.

Requirements — Create .github/workflows/ci-pipeline.yml with the following jobs:

Job 1: lint (Code Quality Check)

- Runs on: ubuntu-latest
- Steps: checkout code, set up Python 3.9, install flake8, run flake8 with the following configuration: max line length 120, ignore E501 (line too long) and W503 (line break before binary operator), fail on any remaining errors
- This job must run on BOTH push to main/develop AND on pull_request events

Job 2: test (Run Unit Tests with MySQL Service Container)

- Runs on: ubuntu-latest
- Needs: lint (must pass first)
- Service container: MySQL 8.0 with health check
- Steps: checkout, set up Python, install dependencies from requirements.txt, wait for MySQL to be healthy, import Sakila schema and data into the service container, run pytest with verbose output
- Environment variables must be set to connect to the MySQL service container

Job 3: build (Docker Image Build & Verification)

- Runs on: ubuntu-latest
- Needs: test (must pass first)
- Steps: checkout, set up Docker Buildx, build the Docker image with the tag sakila-app:${{ github.sha }}, verify the image was created using docker images, run a basic smoke test by starting the container and checking it responds on port 5000 (curl or wget)

Job 4: security-scan (Image Security Scan)

- Runs on: ubuntu-latest
- Needs: build
- Steps: checkout, build the image again (or use Docker layer caching), run Trivy vulnerability scanner (aquasecurity/trivy-action) against the built image, upload the scan results as a workflow artifact
- This job should NOT fail the pipeline on vulnerabilities (use exit-code: 0) but MUST report them

Pipeline Configuration Requirements:

49. The pipeline must trigger on: push to main and develop branches, pull_request to main branch.
50. Use environment variables at the workflow level for common values (Python version, image name).
51. The test job must use GitHub Actions service containers to run MySQL (not Docker Compose).
52. Add a concurrency group to cancel in-progress runs when a new commit is pushed to the same branch.

CHALLENGE ELEMENTS

The MySQL service container in GitHub Actions does NOT automatically load the Sakila database. You must figure out how to download and import the Sakila SQL files as part of your test job steps.

Service containers in GitHub Actions have different network connectivity than local Docker. The MySQL host is 'localhost' in Actions (not a container name). Your pipeline must handle this.

The Trivy security scan will almost certainly find vulnerabilities in the Python base image. You must NOT fail the build on these, but you MUST upload the report.

DELIVERABLES FOR C1

.github/workflows/ci-pipeline.yml committed to your repository

Screenshot of the pipeline running successfully on GitHub Actions (showing all 4 jobs)

Screenshot of the lint job logs (showing flake8 output)

Screenshot of the test job logs (showing pytest results)

Screenshot of the build job logs (showing Docker build and smoke test)

Screenshot of the security-scan job artifact (Trivy report)

Task C2: Conditional Deployment Workflow [8 Marks]

Scenario: You need a separate deployment workflow that only runs when code is merged to main (not on every push). This workflow simulates a deployment process with environment-specific configurations.

Requirements — Create .github/workflows/deploy.yml:

53. Trigger: workflow_run (triggered AFTER the CI pipeline completes successfully on main) OR on push to main with a path filter that only triggers when application code changes (not README or docs).
54. Define TWO environments: staging and production.
55. Job 1 (deploy-staging): simulates staging deployment by printing environment variables, running the Docker image with a staging tag, and outputting a deployment summary using GitHub Actions Job Summary (GITHUB_STEP_SUMMARY).
56. Job 2 (deploy-production): needs deploy-staging, uses the GitHub Actions environment protection rule (add a manual approval requirement for production by using the environment keyword). This job should print a simulated production deployment message and create a GitHub Release with a version tag derived from the current date (e.g., v2026.04.15).
57. Both jobs must include a notification step that uses a run command to echo a simulated notification (e.g., "Slack notification: Deployment to staging successful").

Path Filtering Configuration:

```
# Your workflow should NOT trigger on changes to:
# - README.md
# - docs/**
# - *.md
# - .gitignore

# It SHOULD trigger on changes to:
# - app.py
# - config.py
# - templates/**
# - static/**
# - Dockerfile
# - requirements.txt
# - docker-compose.yml
```

DELIVERABLES FOR C2

.github/workflows/deploy.yml committed to your repository
Screenshot of the deployment workflow running on GitHub Actions
Screenshot of the Job Summary output for staging deployment
Screenshot of the environment protection rule configuration (if applicable)
Screenshot or link to the GitHub Release created by the pipeline

Task C3: Pipeline Debugging Challenge [5 Marks]

Scenario: A broken CI/CD pipeline YAML file has been provided below. It contains 5 distinct errors. You must identify each error, explain why it fails, and provide the corrected version.

The Broken Pipeline (save as .github/workflows/broken-pipeline.yml):

```
name: Broken CI Pipeline

on:
  push:
    branches: main

jobs:
  test:
    runs-on: ubuntu
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: 3.9
      - name: Install deps
        run: |
          pip install -r requirements.txt
          pip install pytest
      - name: Run Tests
        run: pytest tests/ -v

  build:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Image
        run: docker build -t myapp .

  deploy:
    runs-on: ubuntu-latest
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy
        run: echo Deploying...
```

Requirements:

58. Identify ALL 5 errors in the YAML above. For each error, state: the line number (approximate), what the error is, why it causes failure, and the corrected code.
59. Create a corrected version of this file as .github/workflows/fixed-pipeline.yml and commit it.
60. Push the broken version first, show the failure in GitHub Actions, then push the fixed version and show it succeeding.

DELIVERABLES FOR C3

Written analysis of all 5 errors (in your report)
.github/workflows/broken-pipeline.yml committed
.github/workflows/fixed-pipeline.yml committed
Screenshot of the FAILED pipeline run
Screenshot of the SUCCESSFUL pipeline run after fixes

5. PART D — Reflection & Architecture Documentation [15 Marks]

This section tests your ability to think critically about the systems you have built and communicate technical concepts clearly.

Task D1: Architecture Diagram [5 Marks]

Requirements:

61. Create a professional architecture diagram (using draw.io, Lucidchart, Excalidraw, or any diagramming tool) that shows the complete system as defined in your docker-compose.yml.
62. The diagram must include: all 4 services (db, app, db-admin, test-runner), the Docker network with labeled connections, the volume mount for the database, port mappings to the host machine, the flow of data between components (arrows with labels), and environment variables flowing into each service.
63. Include a SECOND diagram showing your CI/CD pipeline stages, their dependencies, and the conditions under which each job runs.

Task D2: Critical Reflection Report [10 Marks]

Write a 2–3 page report addressing the following questions:

64. Git Workflow Analysis (1 paragraph): Why is a branching strategy important in a team environment? Compare Git merge vs. Git rebase — when would you use each, and what are the risks of each approach?
65. Docker Optimization Justification (1–2 paragraphs): Explain every optimization you made in your Dockerfile and WHY each one matters. What is the significance of Docker layer caching and how does COPY ordering affect build times? Why should sensitive data never be hardcoded in a Dockerfile?
66. Networking Analysis (1 paragraph): Explain the difference between Docker's default bridge network and a user-defined bridge network. Why can containers on the default bridge NOT communicate by name? What DNS mechanism does Docker use for user-defined networks?
67. Docker Compose vs. Manual Commands (1 paragraph): What specific problems does Docker Compose solve that manual docker run commands do not? Explain what happens when you run docker compose down -v versus docker compose down — which data is preserved and which is destroyed?
68. CI/CD Pipeline Design Decisions (1–2 paragraphs): Why did you order your pipeline jobs the way you did? What is the benefit of failing fast (lint before test before build)? Why is

it important that the security scan does not block the pipeline? How does concurrency grouping prevent wasted resources?

69. Production Readiness Assessment (1 paragraph): If this application were being deployed to actual production, what additional DevOps practices would you implement? Discuss at least 3 practices from: secrets management, monitoring/observability, container orchestration (Kubernetes), infrastructure as code, or disaster recovery.

GRADING STANDARDS FOR PART D

Surface-level answers will receive minimal marks. You are expected to demonstrate UNDERSTANDING, not just recall.

Use specific examples from YOUR work in this assignment to support your arguments.

Generic answers that could apply to any project will score lower than answers that reference specific decisions you made.

Appendix A: Useful Commands Reference

Git Commands

```
git clone <url>                      # Clone a repository
git checkout -b <branch>             # Create and switch to a new branch
git merge <branch>                   # Merge a branch into current branch
git rebase <branch>                  # Rebase current branch onto another
git log --graph --oneline --all      # Visualize branch history
git tag -a v1.0.0 -m "msg"           # Create annotated tag
git remote add upstream <url>        # Add upstream remote
git stash / git stash pop            # Temporarily save changes
```

Docker Commands

```
docker build -t <name> .                      # Build an image
docker run -d --name <n> --network <net> <img>  # Run container
docker exec -it <container> bash              # Enter a container
docker network create <name>                  # Create a network
docker network inspect <name>                 # Inspect a network
docker volume create <name>                   # Create a volume
docker volume inspect <name>                  # Inspect a volume
docker compose up -d                          # Start all services
docker compose down -v                        # Stop and remove volumes
docker compose ps                             # List running services
docker compose logs -f                        # Follow aggregated logs
docker compose --profile test run <svc>       # Run profiled service
```

GitHub Actions YAML Structure

```
name: Pipeline Name
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
```

```
job-name:
  runs-on: ubuntu-latest
  needs: [dependency-job]
  services:
    mysql:
      image: mysql:8.0
      env:
        MYSQL_ROOT_PASSWORD: admin
      ports:
        - 3306:3306
      options: >-
        --health-cmd="mysqladmin ping -h localhost"
        --health-interval=10s
        --health-timeout=5s
        --health-retries=5
  steps:
    - uses: actions/checkout@v4
    - name: Step name
      run: command
```

Appendix B: Sakila Database Quick Reference

The Sakila database models a DVD rental store. Key tables you may interact with:

Table	Description	Key Columns
actor	People who appear in films	actor_id, first_name, last_name
film	Movies available for rent	film_id, title, description, release_year
customer	People who rent DVDs	customer_id, first_name, last_name, email
rental	Individual rental transactions	rental_id, rental_date, customer_id, inventory_id
inventory	Physical copies of films per store	inventory_id, film_id, store_id
payment	Payments for rentals	payment_id, customer_id, amount, payment_date
store	Physical rental locations	store_id, manager_staff_id, address_id

END OF ASSIGNMENT HANDOUT

Good luck! Remember: the goal is learning, not just completing tasks.